

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION COMMUNICATION TECHNOLOGY

Project Report

Cinema Management System

Group members:

Hoang Manh Cuong - 20226018

Nguyen Tien Dat - 20226023

Supervised by:

Professor Nguyen Thi Oanh

Capstone Project for IT3290E

Hanoi University of Science and Technology

December 30, 2024

Contents

1	Introduction	2
1.1	Problem	2
1.2	Project Scope	2
1.3	Team Organization	2
2	Database Design	2
2.1	Design Process	2
2.2	Entity-Relation Diagram	3
2.3	Relational Diagram	5
3	Triggers and constraints	7
3.1	Constraints	7
3.2	Triggers	8
4	Functions	13
4.1	User's function	14
4.2	Staff's function	17
4.3	Manger's function	17
5	Index analysis	23
6	View	25
7	Application	29
8	Result/Conclusion	32
8.1	Difficulties during Project Execution	32
8.2	Lesson Learned	33

1 Introduction

1.1 Problem

The cinema operates within the entertainment industry, providing movie screenings and additional services (e.g., snacks, beverages) in physical cinema locations. The project aims to offer a smooth ticket booking experience for its guests through an application that helps staff and managers efficiently manage ticket orders, schedules, and customer feedback.

1.2 Project Scope

The scope of this project includes the friendly interface, development, and implementation of a comprehensive database for the online movie ticket booking application. The database will manage all aspects of the application, including user information, movie details, show schedules, ticket bookings and snack-beverage products along with movie tickets. Our project don't focus on the process to booking ticket, we aim to manage the ticket, information to analyze.

1.3 Team Organization

Hoang Manh Cuong 20226018	Nguyen Tien Dat 20226023
Database design: ER schema and Relational Diagram	
Write functions for users/staff/manager	
Write report/slide	
Data processing	Create tables
Build GUI	Write triggers/constraints
Create View/Index	

2 Database Design

2.1 Design Process

Upon the task of designing the database, our team has spent a considerable amount of time to:

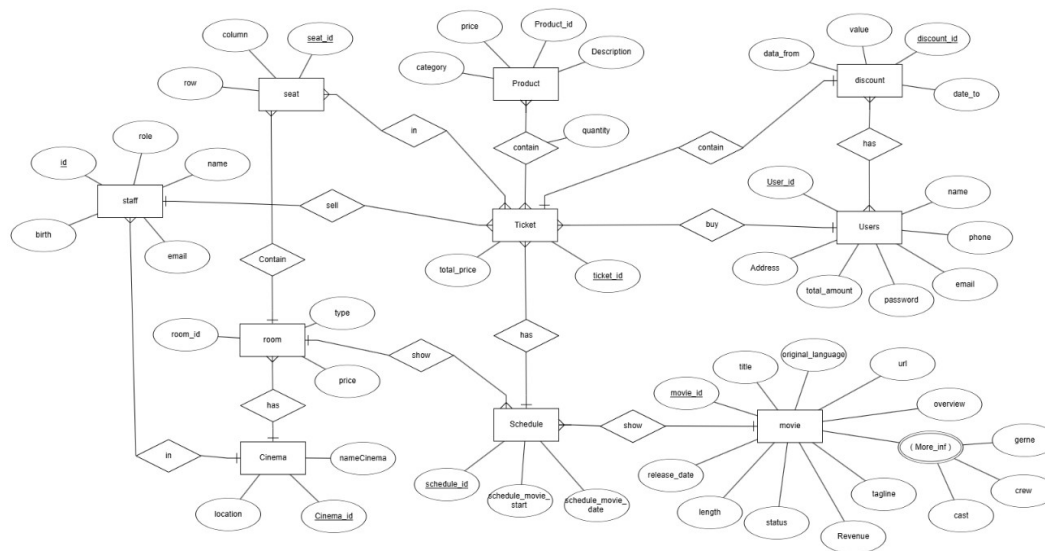
- Brainstorm what is needed.

- Discussing with each other.
- Getting feedback from the instructor.
- Referencing existing systems and resources from the website.

2.2 Entity-Relation Diagram

To design the diagram, we followed a top-down approach. Using the top-down approach for ERD design can help ensure that the resulting database accurately represents the system being modeled, while also making it easier to maintain and update in the future.

2.2.1 ER Diagram



2.2.2 Entity description

- **Movie table:** store all the information: title, overview, length, language, release date, total revenue... of the movie in system. This table has the multi-valued attribute is more_inf which can comprise information about genre, crew, cast.
- **Cinema table:** store name, location of each the cinema in the system.
- **Room table:** store information like: type, price of each room in the cinema.
- **Seat table:** store the information like: row, column of the seat in the corresponding room.
- **Users table** store the information of customer which are name, phone, username, password, email, total amount of money spending in the system of the customer who wants to buy a ticket.

- **Staff table:** store information role, name, email, birth of each staff of each cinema in the system.
- **Product table:** store information, description and price of each product will be sold in the system.
- **Discount table:** store the information about value of discount, this discount can be used by who and when it can be used or expired in system.
- **Schedule table:** store the information comprise the timestamp of the schedule includes date and when the movie showed.
- **Ticket table:** store the total value of the ticket order by the customer.

2.2.3 Relation description

- **Movie - Schedule:** movie have the relation 1 to many with a schedule where one movie can be shown in many schedules. Schedule table will store the ID of movie shown in that schedule.
- **Cinema - Staff:** each cinema can have many staff so the relation is 1 to many where one staff just belong to one cinema. In staff tale will store the information of the cinema they work.
- **Room - Cinema:** each cinema can have many different rooms. So the relation here is 1 to many. In room table, the information of which cinema this room belong.
- **Room - Schedule:** each schedule will be held in one room but one room can have many schedules. So the relation between schedule and room is one to many. Schedule table will store the room this schedule take place.
- **Seat - Room:** each room can comprise many seats. So the relation between seat and room is 1 to many. In seat table will store the information of which room it belongs to.
- **Seat - Ticket:** each ticket can book some seats and one seat can be booked by many tickets with different scheduled show times. So the relation is many to many. And in this relation, we store the price of the seat which is booked by that ticket.
- **Users - Ticket:** each ticket can be bought by one person and each person can buy many tickets. So the relation between ticket and users is one to many. In ticket will store information of users who buy this ticket.

- **Staff - Ticket:** each ticket will be sold by one staff and one staff can sell many tickets. So the relation is one to many. In ticket table, we store information of the staff who sell this ticket.
- **Product - Ticket:** each ticket can buy many products and each product can be bought in many tickets so the relation between product and ticket is many to many. In this relation, we store information of quantity of the products which are bought by that ticket.
- **Discount - Ticket:** each ticket can apply up to 1 discount and each discount can be used for one or many tickets. So the relation between ticket and discount is up to 1 to many. The information of which discount applied will be stored in the ticket table.
- **Discount - Users** each discount can be used by one or many users. Each user can use many available discounts. The information of which user or anyone can use the discount will store in discount table.
- **Schedule - Ticket:** each schedule can show one movie but one movie can be shown by many schedules. So the relation between schedule and movie is one to many. Ticket table will store the schedule of this ticket.

2.3 Relational Diagram

Based on the ER schema and details, we bring out the Relational Diagram below.

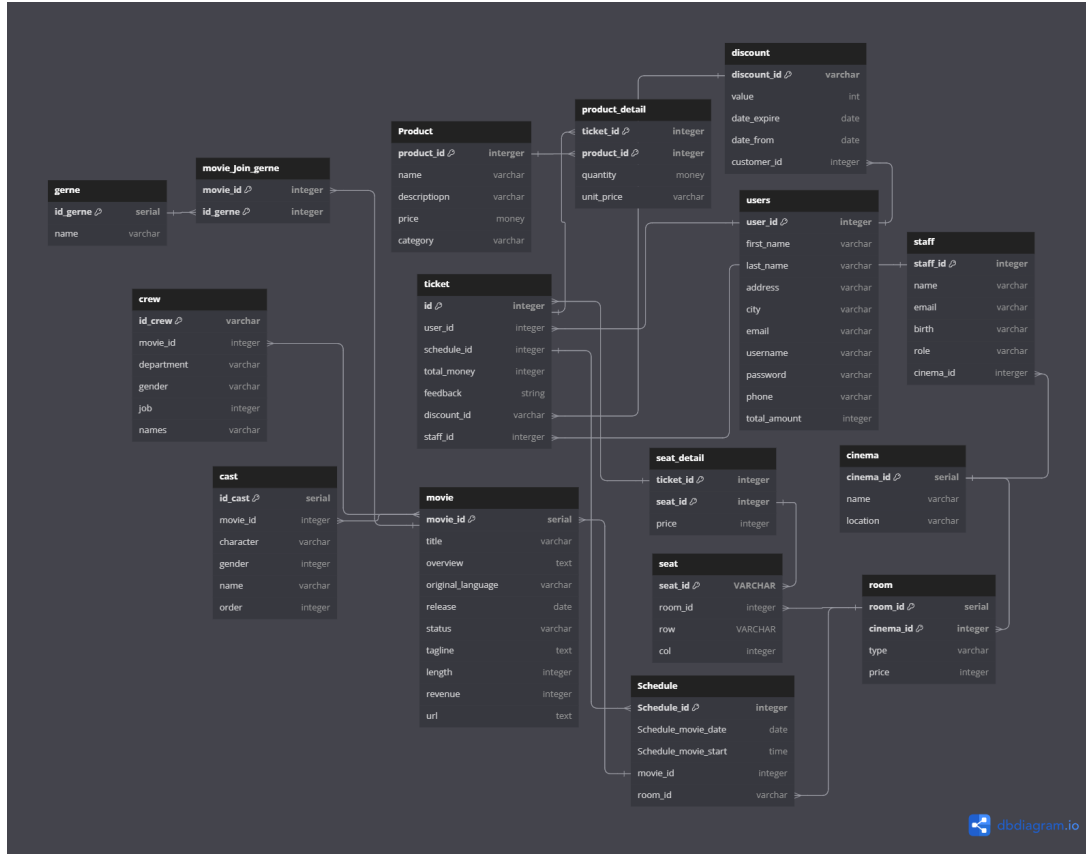


Figure 1: Relational diagram

The process of creating a relational schema involves identifying the entities that need to be represented in the database, defining their attributes, and creating tables representing each entity. We created the relations by following steps:

1. **Identifies the entity:** Each entity or relationship will correspond to a table in the relational schema, and each attribute will be represented by a column in this table.
 ⇒ Based on the ERD above, we create 16 tables for our system.
2. **Create tables for each entity and relationship:** After identifying entities and relationships, we create tables identified from the previous step. An important points is that the table created from relationships must include just two fields (two Primary key with corresponding adjacency entities)
3. **Normalize the schema:** After creating tables, we need to ensure that the schema is normalized to avoid redundancy and ensure data integrity. This involves decomposing the relations into simpler ones that satisfy normal forms as follows:
 - Users - Ticket: Their relationship is One-Many and we know that the ticket_id of Buy table is unique. So ticket_id is considered as **primary key** of Buy table, we can merge them into one table (Add a column as foreign key user_id into the Ticket table and drop Buy table). Users table is kept original. The

same process was applied to relationships between Ticket and 2 difference table (Schedule, Staff). And also applied to staff-cinema, room-schedule, cinema-room, seat-room, schedule-movie.

- Ticket - Seat: Since their relationship is many-to-many so in the relationship In (In table), no field can be considered as **key**. Buut the combination of Ticket_id and Seat_id is unique, so we can not merge to any table. We just renamed the In table $\Rightarrow$ Seat_detail. The same process applies to the relation ticket - product (We just renamed the contain table to product_detail).
- Multivalued attributes such as (genre, crew, cast) hold a seperate table with corresponding **foreign key**.

3 Triggers and constraints

The process of identifying the needed constraints and triggers involves analyzing the requirements and determining what rules need to be enforced and what tasks need to be automated.

3.1 Constraints

Constraints are the rules that we can apply on the type of data in a table. That is, we can specify the limit on the type of data that can be stored in a particular column in a table using constraints. We define the constraints as follows:

3.1.1 Check

a. Check price of a room

Since the price of each room is manually set. It can easily come up a mistake to make the price of the room become too high by add redundant number. We have to check if the price of the room is suspicious, we may not permit to set.

```
1 ALTER TABLE room ADD CONSTRAINT check_room_price
2 CHECK (price > 0 AND price < 500000);
3
```

b. Check valid time of discount

When set date_start or date_expire. There can be an error when manager set mismatch these two attributes. So we have a check to check if the time is valid or not.

```
1 ALTER TABLE discount ADD CONSTRAINT check_valid_date_discount
2 CHECK (date_expire > current_date AND date_expire > date_start);
3
```


3.2 Triggers

SQL triggers are stored procedures that automatically execute in response to certain events in a specific table or view in a database. They are used to maintain the integrity of the data, enforce business rules, and automate tasks. Implementing SQL triggers can significantly improve our database management system.

a. Trigger auto set price seat of seat_detail

When a record is insert into seat_detail table with value of seat's ID and ticket's ID, we will check 2 conditions. Firstly, the ID of seat have to in the room which have the schedule is bought by ticket. Second, when the seat is exactly in the room of that schedule, we will check if this seat have been booked or not. If not, the record successful inserted and auto update the price of seat by the value price of the room.

```
1 CREATE TRIGGER tg_on_update_seat_price
2 BEFORE INSERT OR UPDATE ON seat_detail
3 FOR EACH ROW
4 EXECUTE PROCEDURE tf_on_update_seat_price();
5
6 CREATE OR REPLACE FUNCTION tf_on_update_seat_price()
7 RETURNS TRIGGER AS
8 $$
9 DECLARE sch int;
10 BEGIN
11     sch := (SELECT schedule_id FROM ticket WHERE ticket_id = NEW.
12             ticket_id);
13     -- Check if exist seat in the corresponding room
14     IF EXISTS (
15         SELECT 1
16         FROM ticket t
17             JOIN schedule sh USING (schedule_id)
18         JOIN seat s ON sh.room_id = s.room_id
19         WHERE s.seat_id = NEW.seat_id AND t.ticket_id = NEW.
20             ticket_id
21     ) THEN
22     -- Check whether the seat is booked or not
23     IF NOT EXISTS (
24         SELECT 1
25         FROM ticket t
26             JOIN schedule sche USING (schedule_id)
27         JOIN seat_detail s USING (ticket_id)
28         WHERE s.seat_id = NEW.seat_id and sche.schedule_id = sch
29     )
30     THEN
31         -- update price from room table
32         NEW.price := (
33             SELECT room.price
```

```

32         FROM seat
33         JOIN room USING (room_id)
34         WHERE seat_id = NEW.seat_id
35     );
36     ELSE RAISE NOTICE 'seat_id = % has already booked ',NEW.seat_id;
37 RETURN NULL;
38 END IF;
39 ELSE
40     RAISE NOTICE 'seat_id = % not found', NEW.seat_id;
41 RETURN NULL;
42 END IF;
43 RETURN NEW;
44 END;
45 $$ LANGUAGE plpgsql;
46

```

b. Trigger auto set price of product of product_detail

When a record is insert into product_detail, we will check if the product's ID exists or not. If it exists, automatically update the price of the product.

```

1 CREATE TRIGGER tg_on_update_product_price
2 BEFORE INSERT OR UPDATE ON product_detail
3 FOR EACH ROW
4 EXECUTE PROCEDURE tf_on_update_product_price();
5
6 CREATE OR REPLACE FUNCTION tf_on_update_product_price()
7 RETURNS TRIGGER AS
8 $$
9 BEGIN
10     -- Check if exist product's ID
11     IF EXISTS (
12         SELECT 1
13         FROM product
14         WHERE product_id = NEW.product_id
15     ) THEN
16         -- update price of product
17         NEW.unit_price := (
18             SELECT product.price
19             FROM product
20             WHERE product_id = NEW.product_id
21         );
22     ELSE
23         RAISE NOTICE 'product_id = % not found', NEW.product_id;
24     END IF;
25     RETURN NEW;
26 END;
27 $$ LANGUAGE plpgsql;

```

28
29

c. Trigger auto update ticket's price

Whenever one of table seat_detail or product_detail is inserted/updated/deleted, the ticket's price corresponding will be updated increased or decreased base on the operation is insert or update or delete.

```
1 CREATE TRIGGER tg_on_update_price_product
2 AFTER INSERT OR UPDATE OR DELETE ON product_detail
3 FOR EACH ROW
4 EXECUTE FUNCTION tf_on_update_price();
5
6 CREATE TRIGGER tg_on_update_price_seat
7 AFTER INSERT OR UPDATE OR DELETE ON seat_detail
8 FOR EACH ROW
9 EXECUTE FUNCTION tf_on_update_price();
10
11 CREATE OR REPLACE FUNCTION tf_on_update_price()
12 RETURNS TRIGGER AS
13 $$
14 BEGIN
15     IF TG_OP = 'INSERT' OR TG_OP = 'UPDATE' THEN
16         -- If insert or update
17         IF TG_TABLE_NAME = 'product_detail' THEN
18             -- If action on table product_detail
19             UPDATE ticket
20             SET total_money = total_money
21                 + COALESCE(NEW.quantity * NEW.unit_price, 0)
22             WHERE ticket_id = NEW.ticket_id;
23
24         ELSIF TG_TABLE_NAME = 'seat_detail' THEN
25             -- If action on table seat_detail
26             UPDATE ticket
27             SET total_money = total_money
28                 + COALESCE(NEW.price, 0)
29             WHERE ticket_id = NEW.ticket_id;
30         END IF;
31
32     ELSIF TG_OP = 'DELETE' THEN
33         -- If delete
34         IF TG_TABLE_NAME = 'seat_detail' THEN
35             -- If action on table seat_detail
36             UPDATE ticket
37             SET total_money = total_money
38                 - COALESCE(OLD.price, 0)
39             WHERE ticket_id = OLD.ticket_id;
```

```

40
41     ELSIF TG_TABLE_NAME = 'product_detail' THEN
42         -- If action on table product_detail
43         UPDATE ticket
44         SET total_money = total_money
45             - COALESCE(OLD.quantity * OLD.unit_price, 0)
46         WHERE ticket_id = OLD.ticket_id;
47     END IF;
48 END IF;
49 RETURN NEW;
50 END;
51 $$ LANGUAGE plpgsql;
52
53

```

d. Trigger for insert discount

When discount is applied to one ticket, after check the valid of discount by the date start and date expire. This trigger will automatically update the price of ticket by the value corresponding in discount table. Or if remove discount, the price of ticket will return back to the previous value.

```

1 CREATE TRIGGER tg_on_discount
2 AFTER UPDATE ON ticket
3 FOR EACH ROW
4 WHEN (NEW.discount_id IS DISTINCT FROM OLD.discount_id)
5 EXECUTE PROCEDURE tf_on_discount();
6
7 CREATE OR REPLACE FUNCTION tf_on_discount()
8 RETURNS TRIGGER AS
9 $$
10 BEGIN
11     IF NEW.discount_id IS NOT NULL THEN
12         IF EXISTS (
13             SELECT 1
14             FROM discount
15             WHERE discount_id = NEW.discount_id
16                 AND date_expire < CURRENT_DATE
17         ) THEN
18             RAISE NOTICE 'discount_id = % is expired!', NEW.
discount_id;
19             RETURN NEW;
20         END IF;
21     IF EXISTS (
22         SELECT 1
23         FROM discount
24         WHERE discount_id = NEW.discount_id
25     ) THEN

```

```

26         UPDATE ticket
27         SET total_money = OLD.total_money * (1 - (
28             SELECT value::float / 100
29             FROM discount
30             WHERE discount_id = NEW.discount_id
31         ))
32         WHERE ticket_id = OLD.ticket_id;
33     ELSE
34         RAISE NOTICE 'discount_id = % does not exist!', NEW.
discount_id;
35     END IF;
36
37     ELSIF OLD.discount_id IS NOT NULL THEN
38         UPDATE ticket
39         SET total_money = OLD.total_money / (1 - (
40             SELECT value::float / 100
41             FROM discount
42             WHERE discount_id = OLD.discount_id
43         ))
44         WHERE ticket_id = OLD.ticket_id;
45     END IF;
46     RETURN NEW;
47 END;
48 $$ LANGUAGE plpgsql;
49

```

e. Trigger update total spending of customers

After one ticket is made, this trigger will add the price of this ticket into the total spending of the corresponding user who own this ticket. Respectively when the ticket is deleted when the staff makes it wrong, the total spending of that user own that ticket will be decrease.

```

1 CREATE TRIGGER tg_insert_on_spending
2 AFTER UPDATE ON ticket
3 FOR EACH ROW
4 EXECUTE PROCEDURE tf_update_on_spending();
5
6 CREATE OR REPLACE FUNCTION tf_update_on_spending()
7 RETURNS TRIGGER AS
8 $$
9 BEGIN
10     UPDATE users
11     SET total_amount = total_amount + NEW.total_money - OLD.
total_money
12     WHERE user_id = NEW.user_id;
13 RETURN NEW;
14 END;

```

```

15 $$ LANGUAGE plpgsql
16

```

f. Check conflict of schedule

In the same timestamp, same room there will be one schedule at a time. So before insert a schedule, we have to check if there will be any conflict between this schedule and all the previous one had inserted. If there no conflict, the record can be added into table.

```

1 CREATE TRIGGER tg_check_conflict_schedule
2 BEFORE INSERT ON schedule
3 FOR EACH ROW
4 EXECUTE PROCEDURE tf_check_conflict_schedule();
5
6 CREATE OR REPLACE FUNCTION tf_check_conflict_schedule()
7 RETURNS TRIGGER AS
8 $$
9 BEGIN
10     IF EXISTS (
11         SELECT 1
12         FROM schedule
13         WHERE schedule_movie_date = NEW.schedule_movie_date
14             AND room_id = NEW.room_id
15             AND (
16                 NEW.schedule_movie_start::time < (
17                     schedule_movie_start::time + interval '3 hours')
18                 AND (NEW.schedule_movie_start::time + interval '3
19                     hours') > schedule_movie_start::time
20             )
21     ) THEN
22         RAISE NOTICE 'Overlapped schedule!!';
23         RETURN NULL;
24     END IF;
25     RETURN NEW;
26 END;
27 $$ LANGUAGE plpgsql;
28

```

4 Functions

A PostgreSQL function or a stored procedure is a set of SQL and procedural commands such as declarations, assignments, loops, flow-of-control etc. stored on the database server and can be involved using the SQL interface. And it is also known as PostgreSQL stored procedures. When writing functions for database, we need an optimized strategy to

ensure that these functions can perform efficiently and easy to maintain. In our project, we will provide functions to three groups (User, Manager, Staff).

4.1 User's function

a. Search movie

(a) Description

When customer want to find out some upcoming showed movie, they will have request to list some movie that satisfy what they want like title of the movie. So we have to define the function for them to used to easily search for the movie they want.

(b) Implementation

```
1 CREATE OR REPLACE FUNCTION search_movie(in names varchar)
2 RETURNS table (
3     Movie_name varchar,
4     overviews text,
5     ori_language varchar,
6     release date,
7     statu varchar,
8     taglines text,
9     length integer
10 ) as
11 $$
12 begin
13     return query
14     SELECT title, overview, original_language, release_date ,
15           status,tagline, runtime
16     FROM movie
17     WHERE title ILIKE '%' || names || '%';
18 END;
19 $$ language plpgsql;
```

b. Show cast of this movie

(a) Description

After they see the movie, They can see more detail about casts of this movie. So we have to define the function for them to see all casts of that movie they want.

(b) Implementation

```
1 CREATE OR REPLACE FUNCTION show_cast(in movie_name varchar)
2 RETURNS TABLE(
3     cast_name varchar,
```

```

4   gender varchar,
5   VaiDien varchar,
6   orders integer,
7   movie_title varchar
8 ) AS
9 $$
10 BEGIN
11   RETURN QUERY
12   SELECT c.name, c.gender, c.character, c.order, m.title
13   FROM "cast" c
14   JOIN movie m ON c.movie_id = m.movie_id
15   WHERE m.title ILIKE '%' || movie_name || '%';
16 END;
17 $$ language plpgsql;

```

c. Show crew of this movie

(a) Description

After they see the movie, They can see more detail about crews of this movie. So we have to define the function for them to see all crews of that movie they want.

(b) Implementation

```

1 CREATE OR REPLACE FUNCTION show_crew(in movie_name varchar)
2 RETURNS TABLE(
3   crew_name varchar,
4   department varchar,
5   gender integer,
6   job varchar,
7   movie_title varchar
8 ) AS
9 $$
10 BEGIN
11   RETURN QUERY
12   SELECT c.name, c.department, c.gender, c.job, m.title
13   FROM crew c
14   JOIN movie m ON c.movie_id = m.movie_id
15   WHERE m.title ILIKE '%' || movie_name || '%';
16 END;
17 $$ language plpgsql;

```

d. List all schedule in day

(a) Description

In order to choose movie to watch, customers have to choose schedule. So we

have to list the schedule upcoming in day for them to decide to choose that schedule or not.

(b) Implementation

```
1 create or replace function schedule_in_day()
2 returns table (
3     title varchar(100),
4     overview text,
5     schedule_movie_date date,
6     schedule_movie_start varchar(10)
7 ) as
8 $$
9 begin
10     return query
11     select m.title, m.overview, s.schedule_movie_date, s.
12           schedule_movie_start
13     from schedule s
14     join movie m using (movie_id)
15     where s.schedule_movie_date >= current_date;
16 end;
17 $$ language plpgsql;
```

e. Search genre

(a) Description

When customer hesitate to choose the movie, they can search for the genre. So we create a function to them search by name of genre, the output this list of movies that having this genre.

(b) Implementation

```
1 CREATE OR REPLACE FUNCTION search_genre(in genre_name varchar)
2 RETURNS TABLE(
3     movie_title varchar,
4     released_date date,
5     genres varchar
6 ) AS
7 $$
8 BEGIN
9     RETURN QUERY
10     SELECT m.title, m.release_date, g.name AS genres
11     FROM movie m
12     JOIN movie_join_genre mjg ON m.movie_id = mjg.movie_id
13     JOIN genre g ON mjg.id_genre = g.id_genre
14     WHERE g.name ILIKE '%' || genre_name || '%'
15     ORDER BY m.release_date DESC;
16 END;
17 $$ language plpgsql;
```

4.2 Staff's function

a. Search ticket

(a) Description

When an issue come up, staff have to check previous for some information. So we have a function to help staff faster to look up ticket by the ticket's ID is the input.

(b) Implementation

```
1 CREATE OR REPLACE FUNCTION search_ticket(in ticketID integer)
2 RETURNS TABLE(
3     ticket_ids integer,
4     usernames varchar,
5     schedule_movie_dates date,
6     schedule_movie_starts varchar,
7     room_names varchar,
8     cinema_names varchar,
9     movie_titles varchar,
10    total_moneys int,
11    feedbacks text,
12    discount_values integer,
13    seat_detail text,
14    product_detail text
15 ) AS
16 $$
17 BEGIN
18     RETURN QUERY
19         SELECT ticket_id, username, schedule_movie_date,
20            schedule_movie_start, room_name, cinema_name, movie_title,
21            total_money, feedback, discount_value, seat_details,
22            product_details
23         FROM view_ticket
24         WHERE ticket_id = ticketID
25         ORDER BY schedule_movie_date, schedule_movie_start;
26 END;
27 $$ language plpgsql;
```

4.3 Manger's function

a. Delete the discount which expired

(a) Description

The discount is not automatically removed when the date is expired. When the manager check if there no need to maintain these discount in the system, the manager will call function to remove.

(b) Implementation

```
1 CREATE OR REPLACE FUNCTION tf_delete_discount_expire()
2 RETURNS VOID AS
3 $$
4 BEGIN
5 DELETE FROM discount WHERE date_expire < CURRENT_DATE;
6 END;
7 $$ LANGUAGE plpgsql;
8
9
```

b. Show history transaction of one customers

(a) Description

This function will return the information of customers and ticket they have bought. With the input value is the username and email.

(b) Implementation

```
1 CREATE OR REPLACE FUNCTION history_trans(usersds VARCHAR(30),
2      gmail VARCHAR(50))
3 RETURNS TABLE(
4     usernames VARCHAR,
5     email VARCHAR,
6     name TEXT,
7     movie VARCHAR,
8     room VARCHAR,
9     total_money INTEGER,
10    feedback TEXT,
11    discount_id VARCHAR,
12    staff_name VARCHAR,
13    schedule_movie_date DATE,
14    schedule_movie_start VARCHAR
15 ) AS
16 $$
17 BEGIN
18     RETURN QUERY
19     SELECT
20         u.username AS usernames,
21         u.email,
22         (u.first_name || ' ' || u.last_name) AS name, --
23         Explicit cast to VARCHAR
24         m.title AS movie,
25         r.name AS room, -- Use the correct room name
26         t.total_money,
27         t.feedback,
28         t.discount_id,
```

```

27         s.name AS staff_name,
28         sc.schedule_movie_date,
29         sc.schedule_movie_start
30     FROM ticket t
31     JOIN schedule sc ON t.schedule_id = sc.schedule_id
32     JOIN movie m ON sc.movie_id = m.movie_id
33     JOIN users u ON t.user_id = u.user_id
34     JOIN staff s ON t.staff_id = s.staff_id
35     JOIN room r ON sc.room_id = r.room_id -- Add a JOIN for the
        room name
36     WHERE u.username = usersds AND u.email = gmail;
37 END;
38 $$ LANGUAGE plpgsql;
39
40

```

c. Search for number booked seat of a schedule

(a) Description

In order for manager to know the cover percentage of one schedule, this function receive schedule's ID as input and return table with the information of timestamp, room's ID and number seat have booked.

(b) Implementation

```

1 CREATE OR REPLACE FUNCTION search_booked_seat(in scheduleid int
    )
2 RETURNS TABLE(
3     room_name varchar,
4     date date,
5     start varchar,
6     number_booked bigint,
7     cover_percentage float
8 ) AS
9 $$
10 DECLARE num_seat int;
11 BEGIN
12     SELECT COUNT(*) INTO num_seat
13 FROM seat
14 JOIN schedule USING (room_id)
15 WHERE schedule_id = scheduleid;
16 RETURN QUERY
17 SELECT r.name, scc.schedule_movie_date, scc.
    schedule_movie_start, COUNT(DISTINCT sd.seat_id) as
    number_booked, ROUND((COUNT(DISTINCT sd.seat_id) * 100) /
    num_seat::float,2) as cover_percentage
18 FROM seat_detail sd

```

```

19 JOIN seat s ON sd.seat_id = s.seat_id
20 JOIN schedule scc ON s.room_id = scc.room_id
21 JOIN room r ON s.room_id = r.room_id
22 WHERE scc.schedule_id = scheduleid
23 GROUP BY r.name, scc.schedule_movie_date, scc.
         schedule_movie_start;
24 END;
25 $$ language plpgsql;
26
27
28

```

d. Display top n customers spending highest

(a) Description

Display all customer spending much money in rank n. Input: n (rank n that manager want to see) out: list of all customer in that rank.

(b) Implementation

```

1 CREATE OR REPLACE FUNCTION vip_customer(IN num INT)
2 RETURNS TABLE (
3     user_id INT,
4     first_name VARCHAR(30),
5     last_name VARCHAR(30),
6     email VARCHAR(50),
7     phone VARCHAR(12),
8     username VARCHAR(30),
9     total_amounts INT,
10    ranks bigint
11 ) AS
12 $$
13 BEGIN
14     RETURN QUERY
15     WITH ranked_users AS (
16         SELECT
17             u.user_id,
18             u.first_name,
19             u.last_name,
20             u.email,
21             u.phone,
22             u.username,
23             u.total_amount,
24             DENSE_RANK() OVER (ORDER BY u.total_amount DESC) AS
rank
25         FROM users u
26     )
27     SELECT

```

```

28         r.user_id,
29         r.first_name,
30         r.last_name,
31         r.email,
32         r.phone,
33         r.username,
34         r.total_amount,
35         r.rank
36     FROM ranked_users r
37     WHERE r.total_amount >= (
38         SELECT MIN(total_amount)
39         FROM ranked_users
40         WHERE rank = num
41     )
42     ORDER BY r.rank;
43 END;
44 $$ LANGUAGE plpgsql;

```

e. Display quantity of product sold in amount of time

(a) Description

Display product name and the quantity corresponding. Inputs are two date, the function will return all the product was sold in that interval of time.

(b) Implement

```

1 CREATE OR REPLACE FUNCTION product_sold(in date_start date, in
    date_end date)
2 RETURNS TABLE(
3     product_name varchar,
4     quantity_sold bigint
5 ) AS
6 $$
7 BEGIN
8     RETURN QUERY
9     SELECT p.name, SUM(pd.quantity)
10    FROM product_detail pd
11   JOIN product p ON pd.product_id = p.product_id
12   JOIN ticket t ON pd.ticket_id = t.ticket_id
13   JOIN schedule s ON t.schedule_id = s.schedule_id
14  WHERE (s.schedule_movie_date >= date_start) AND (s.
    schedule_movie_date <= date_end)
15  GROUP BY p.name;
16 END;
17

```

f. Display total ticket was sold until now

(a) **Description**

Input of this function is a point of time. This function will return all movie was showed from that time and the number of ticket sold corresponding from that timepoint until now.

(b) **Implement**

```
1 CREATE OR REPLACE FUNCTION ticket_sold(in start_date date)
2 RETURNS TABLE(
3     Start_dates date,
4     total_ticket_sold bigint,
5     movie_name varchar
6 ) AS
7 $$
8 BEGIN
9     RETURN QUERY
10    SELECT s.schedule_movie_date, COUNT(t.ticket_id), m.title
11    FROM ticket t
12    JOIN schedule s ON t.schedule_id = s.schedule_id
13    JOIN movie m ON s.movie_id = m.movie_id
14    WHERE s.schedule_movie_date = start_date
15    GROUP BY s.schedule_movie_date, m.title;
16 END;
17 $$ language plpgsql;
18
```

g. **Display total revenue in the interval of time**

(a) **Description**

Inputs of this function are two point of time, then the function will return the total revenue over all movie in that interval of time.

(b) **Implement**

```
1 CREATE OR REPLACE FUNCTION revenue_period(in date_start date,
2     in date_end date)
3 RETURNS TABLE(
4     total_revenue bigint ,
5     date_starts date,
6     date_ends date
7 ) AS
8 $$
9 BEGIN
10    RETURN QUERY
11    SELECT SUM(t.total_money), date_start, date_end
12    FROM ticket t
13    JOIN schedule s ON t.schedule_id = s.schedule_id
```

```

13 WHERE s.schedule_movie_date >= date_start AND s.
    schedule_movie_date <= date_end;
14 END;
15 $$ language plpgsql;
16

```

h. Display the revenue of one movie until recent

(a) Description

Input for the function is the movie's ID, then the function will return the revenue of the movie from the time it show until now.

(b) Implement

```

1 create or replace function total_revenue_movie(in movieid int)
2 returns bigint as
3 $$
4 declare total_rev bigint;
5 begin
6 select sum(seat_detail.price) into total_rev
7 from ticket t
8 join seat_detail using (ticket_id)
9 join schedule using (schedule_id)
10 join movie using (movie_id)
11 where schedule.movie_id = movieid;
12
13 update movie
14 set revenue = total_rev
15 where movie_id = movieid;
16 return total_rev;
17 end;
18 $$ language plpgsql
19
20

```

5 Index analysis

1. **Index on schedule** Because the query to see the schedule is used more than the insert, we create index Btree for this

QUERY PLAN
text
Sort (cost=92.25..92.26 rows=1 width=142) (actual time=0.205..0.205 rows=0 loops=1)
Sort Key: s.schedule_movie_date, s.schedule_movie_start
Sort Method: quicksort Memory: 25kB
-> Nested Loop (cost=0.58..92.24 rows=1 width=142) (actual time=0.200..0.200 rows=0 loops=1)
-> Nested Loop (cost=0.30..83.94 rows=1 width=130) (actual time=0.200..0.200 rows=0 loops=1)
-> Nested Loop (cost=0.15..83.71 rows=1 width=56) (actual time=0.200..0.200 rows=0 loops=1)
-> Seq Scan on schedule s (cost=0.00..75.52 rows=1 width=18) (actual time=0.199..0.200 rows=0 loop...
Filter: (schedule_movie_date >= CURRENT_DATE)
Rows Removed by Filter: 3501
-> Index Scan using room_pkey on room r (cost=0.15..8.17 rows=1 width=46) (never executed)
Index Cond: (room_id = s.room_id)
-> Index Scan using cinema_pkey on cinema c (cost=0.15..0.23 rows=1 width=82) (never executed)
Index Cond: (cinema_id = r.cinema_id)
-> Index Scan using movie_id_index on movie m (cost=0.28..8.30 rows=1 width=20) (never executed)
Index Cond: (movie_id = s.movie_id)
Planning Time: 0.919 ms
Execution Time: 0.226 ms

(a) Before indexing

QUERY PLAN
text
1 Nested Loop (cost=0.86..25.03 rows=1 width=142) (actual time=0.021..0.022 rows=0 loops=1)
2 -> Nested Loop (cost=0.58..16.72 rows=1 width=130) (actual time=0.021..0.021 rows=0 loops=1)
3 -> Nested Loop (cost=0.43..16.40 rows=1 width=56) (actual time=0.021..0.021 rows=0 loops=1)
4 -> Index Scan using idx_schedule_data_start on schedule s (cost=0.28..8.30 rows=1 width=18) (actual time=0.020..0.021 rows=0 loop...
5 Index Cond: (schedule_movie_date >= CURRENT_DATE)
6 -> Index Scan using room_pkey on room r (cost=0.15..8.17 rows=1 width=46) (never executed)
7 Index Cond: (room_id = s.room_id)
8 -> Index Scan using cinema_pkey on cinema c (cost=0.15..0.23 rows=1 width=82) (never executed)
9 Index Cond: (cinema_id = r.cinema_id)
10 -> Index Scan using movie_id_index on movie m (cost=0.28..8.30 rows=1 width=20) (never executed)
11 Index Cond: (movie_id = s.movie_id)
12 Planning Time: 1.461 ms
13 Execution Time: 0.057 ms

(b) After indexing

Figure 2: Comparison of before and after indexing

This shows that the execution time is reduce from 0.226 ms to 0.057 ms.

- Before index: The filter `schedule_movie_date >= CURRENT_DATE` must scan the entire schedule table to filter the relevant rows. Rows Removed by Filter: 3501 — this means that many irrelevant rows were discarded after the sequential scan.
- Index Scan on the schedule table: The condition `schedule_movie_date >= CURRENT_DATE` is optimized using an index. Instead of scanning the entire table, the index significantly reduces the number of rows scanned.

2. Index on movie name

QUERY PLAN
text
1 Seq Scan on movie (cost=0.00..380.04 rows=1 width=521) (actual time=1.011..4.382 rows=9 loops=1)
2 Filter: ((title)::text ~* "%Ava%::text)
3 Rows Removed by Filter: 4794
4 Planning Time: 0.854 ms
5 Execution Time: 4.401 ms

(a) Before indexing

QUERY PLAN
text
1 Bitmap Heap Scan on movie (cost=12.82..16.83 rows=1 width=521) (actual time=0.018..0.031 rows=9 loops=1)
2 Recheck Cond: (title)::text ~* "%Ava%::text)
3 Heap Blocks: exact=9
4 -> Bitmap Index Scan on idx_movie_overview_trgm (cost=0.00..12.82 rows=1 width=0) (actual time=0.011..0.011 rows=9 loop=1)
5 Index Cond: (title)::text ~* "%Ava%::text)
6 Planning Time: 1.613 ms
7 Execution Time: 0.056 ms

(b) Gin Index

Figure 3: Comparison of before and after indexing

Problem Statement: When searching for movie titles in a database, users might:

- Remember only part of the movie's name.
- Enter names with typos or case mismatches.
- For example: Searching for "avengers" instead of "The Avengers". Typing "satr wars" instead of "Star Wars".

By implementing `pg_trgm` and creating a GIN index with `gin_trgm_ops`, we can optimize searches and handle partial or fuzzy matches efficiently.

- Trigrams: These are subsets of 3 consecutive characters extracted from a string.

- For instance, the word "action" is broken down into the following trigrams: ['act', 'act', 'cti', 'tio', 'ion', 'on ']. (Spaces are added to the beginning and end of the string to support edge cases.)
- PostgreSQL uses these trigrams to build an index, which significantly speeds up searches across long strings or text fields.

Comparasion:

(a) **Before Index:**

- Without the GIN index, PostgreSQL performs a sequential scan, which scans every row in the movie table.
- The query needs to examine and filter all rows, resulting in a significantly higher execution time as table size increases.

(b) **After Index:**

- With the GIN index, PostgreSQL quickly identifies relevant rows via the idx_movie_overview_trgm index and avoids scanning the entire table.
- Execution time is drastically reduced, even with a slightly increased planning time

3. Index on total revenue:

QUERY PLAN	
text	
1	Sort (cost=9427.73..9432.73 rows=2001 width=106) (actual time=32.239..32.306 rows=2001 loops=1)
2	Sort Key: total_amount
3	Sort Method: quicksort Memory: 311kB
4	-> Seq Scan on users (cost=0.00..9318.01 rows=2001 width=106) (actual time=0.008..31.796 rows=2001 loops=1)
5	Planning Time: 0.866 ms
6	Execution Time: 32.413 ms

(a) Before index

QUERY PLAN	
text	
1	Index Scan using idx_total_revenue on users (cost=0.28..7283.93 rows=2001 width=106) (actual time=0.027..0.830 rows=2001 loops=1)
2	Planning Time: 1.415 ms
3	Execution Time: 0.885 ms

(b) After Index

Figure 4: Comparison of before and after indexing

By using the index btree on the user.totalamount of that make the query reduce the cost from 9427.73..9432.73 to 0.28..7283.93 and execution time from 32.413 ms to 0.0885 ms

Otherwise, we also make the Gin index in the genre.name for search by name of genre and the output is the list of movies. (See in source code.)

6 View

A **view** in SQL terminology is a single table that is derived from other table. These tables can be *base tables* or previously defined views. A view does not neccessary exist

in physical form; it is considered to be a **virtual table**. View can be use to hide some of the columns from the table, so that increases the security. We defined some view for our project as follows:

1. View schedule

(a) Description

We provide a schedule with some important information: schedule date, start, room, cinema, movie.

(b) Implementation

```
1 CREATE OR REPLACE VIEW view_schedule AS
2     SELECT S.schedule_id, S.schedule_movie_date, s.
       schedule_movie_start, r.name AS room_Name, c.name AS
       cinema_name, m.title AS movie_title
3     FROM schedule S
4     JOIN room R ON S.room_id = R.room_id
5     JOIN cinema C ON R.cinema_id = C.cinema_id
6     JOIN movie M ON S.movie_id = M.movie_id
7     ORDER BY S.schedule_movie_date, s.schedule_movie_start;
```

2. View Ticket

(a) Description

We make a view for see useful information including username, email, schedule date, start, room, movie, cinema, total money, feedback, details about seats and product. By creating this view, we can conceal some sensitive information like movie_id, room_id, cinema_id, user_id.

(b) Implementation

```
1 CREATE OR REPLACE VIEW view_ticket AS
2 SELECT  t.ticket_id,
3         u.username,
4         u.email,
5         s.schedule_movie_date,
6         s.schedule_movie_start,
7         r.name AS room_name,
8         c.name AS cinema_name,
9         m.title AS movie_title,
10        t.total_money,
11        t.feedback,
12        d.value AS discount_value,
13        STRING_AGG(DISTINCT sd.seat_id || ' (' || sd.price || '
14        )', ', ') AS seat_details,
15        STRING_AGG(
```

```

15         DISTINCT prd.name || ' (' || p.unit_price || ' x '
        || p.quantity || ')',
16         ', '
17     ) AS product_details
18 FROM ticket t
19 JOIN users u ON t.user_id = u.user_id
20 JOIN schedule s ON t.schedule_id = s.schedule_id
21 JOIN room r ON s.room_id = r.room_id
22 JOIN cinema c ON r.cinema_id = c.cinema_id
23 JOIN movie m ON s.movie_id = m.movie_id
24 LEFT JOIN discount d ON t.discount_id = d.discount_id
25 LEFT JOIN seat_detail sd ON t.ticket_id = sd.ticket_id
26 LEFT JOIN product_detail p ON t.ticket_id = p.ticket_id
27 LEFT JOIN product prd ON p.product_id = prd.product_id
28 GROUP BY
29     t.ticket_id, u.username, u.email, s.schedule_movie_date
30     ,
31     s.schedule_movie_start, r.name, c.name, m.title,
32     t.total_money, t.feedback, d.value
33 ORDER BY
34     s.schedule_movie_date, s.schedule_movie_start

```

3. View Revenue Per Day

(a) Description

we create a view for providing the revenue of each day for each cinema. It will give the overview of revenue for Senior Manager and for analysis. For example, Manager can compare the revenue of each cinema.

(b) Implementation

```

1 CREATE OR REPLACE VIEW RevenuePerDay AS
2     SELECT
3         s.schedule_movie_date,
4         r.cinema_id,
5         SUM(t.total_money) AS total_revenue
6     FROM ticket t
7     JOIN schedule s ON t.schedule_id = s.schedule_id
8     JOIN room r ON s.room_id = r.room_id
9     GROUP BY s.schedule_movie_date, r.cinema_id
10    ORDER BY s.schedule_movie_date;

```

4. View The Product sale

(a) Description

We create a view to observe the total quantity of products sold, enabling further analysis for various purposes.

(b) Implementation

```
1 CREATE OR REPLACE VIEW ProductSalesView AS
2 SELECT
3     p.product_id,
4     p.name AS product_name,
5     SUM(pd.quantity) AS total_quantity_sold,
6     SUM(pd.unit_price) AS total_revenue
7 FROM product p
8 JOIN product_detail pd ON p.product_id = pd.product_id
9 GROUP BY p.product_id, p.name;
```

5. View for cast, crew, user (with conceal private information)

(a) Description

This view to show information of all cast, crew, user for Customer and Staff, and conceal some private information like phone, email, id, password, user-name

(b) Implementation

```
1 --View of crew
2 CREATE OR REPLACE VIEW crew_details AS
3     SELECT name AS "Name", department, gender, job , movie.
4         title AS movie_title, movie.release_date
5     FROM crew
6     JOIN movie ON crew.movie_id = movie.movie_id;
7
8 --View of cast
9 CREATE OR REPLACE VIEW cast_details AS
10    SELECT c.name AS Name, c.gender, c.character, c.order,
11        movie.title AS movie_title, movie.release_date
12    FROM "cast" c
13    JOIN movie ON c.movie_id = movie.movie_id;
14
15 --Q8 View of user
16 CREATE OR REPLACE VIEW user_summary AS
17    SELECT first_name || ' ' || last_name AS full_name, email,
18        phone, total_amount
19    FROM users
20    ORDER BY total_amount DESC;
```

6. View staff's summary information

(a) Description

For manager to observe all the staff of one cinema, this view contain the information of all staffs who belong to corresponding cinema in the system.

(b) **Implement**

```
1 CREATE VIEW StaffSummaryView AS
2 SELECT
3     c.name AS cinema_name ,
4     st.staff_id ,
5     st.name AS staff_name ,
6     st.email ,
7     st.role
8 FROM cinema c
9 JOIN staff st ON c.cinema_id = st.cinema_id
10 ORDER BY c.name ;
11
```

7. View user's summary information

(a) **Description**

This view has short information of users and the total spending of users in system.

(b) **Implementation**

```
1 CREATE OR REPLACE VIEW cast_details AS
2 SELECT c.name AS Name, c.gender, c.character, c.order, movie.
3     title AS movie_title, movie.release_date
4 FROM "cast" c
5 JOIN movie ON c.movie_id = movie.movie_id
```

7 Application

We also provide a user interface demo for the Manager. Our GUI is built by Python and Tkinter library.

Our login:

Login

Cinema management system

LOGIN

Username

Password

Login

☐ Remember Me

Main Screen

Cinema Management System

Dashboard

Movie

Customer

Ticket

Schedule

Our mainscreen:

Main Screen

Cinema Management System

Dashboard

Name	Email	Phone	Longtlen	Address	City	Username
Hoang Cuong	hoangmanhcuong29@gmail.com	083-365-2146	Longlien	Hanoi	cuonghoang223	
Salina Knavel	salina.knavel@gmail.com	613-280-7170	4 Newark St	Annaprior	dwilberfh	
Yong Yestramski	yong_yestramski@aol.com	519-344-3979	363 Zebra Pl	Brentford	legleme	
Sheron Louissart	sheron@aol.com	718-976-8610	97 E 3rd St #9	Long Island City	None	
Ranee Maye	raanee_maye@maye.com	450-700-5970	5361 N 300 W #3005	Les Cedres	dhalleyrd	
Truman Feichtne	tfeichtne@yahoo.com	973-852-2236	539 Caldwate Canyon Ave	Bloomfield	None	
Ty Smith	tsmith@aol.com	201-570-1153	4546 Kaskhanna St	Hackensack	None	
Kathi Ewart	kwart@icloud.com	906-229-5393	973 El Camino Real	Dorby Junction	stiffj	
Shantell Lizama	shantellizama@gmail.com	07-5346-5917	9787 Dunkelfory Rd	Logan Village	zithanetcl	
Luzon Michon	luzon@cox.net	514-578-7262	5 Goodard Rd	Montreal	sculph	
Shalon Shadrick	shalon@cox.net	718-232-2337	61047 Mayfield Ave	Brooklyn	None	
Suzana Baumgart	suzana.baumgart@yahoo.com	02-5410-5137	7 Elm Ave	Yanico	velbrodwayfd	
Kimberla Dumas	kimberla_dumas@yahoo.com	785-629-8542	8100 Jacksonville Rd #7	Hays	None	
Ima Wolgramm	ima.wolgramm@hotmail.com	973-545-7155	9745 W Main St	Randolph	None	
Patti Rempel	patti@rempe.co.uk	01539-157435	1346 Vandyske St	Hove Park Ward	aantoshin98	
Katlyn Filcroft	kfilcroft@hotmail.com	07-1778-9968	7177 E 14th St	Makery	bandrich1e	
Miss Dunning	mdunning@gmail.com	306-508-8874	38 Douglas Rd E #39	Regina	dcheaney44	
Avery Steier	avery@cox.net	407-808-9439	93 Richmond Rd #402	Orlando	None	
Ettie Hoopengardner	ettie.hoopengardner@hotmail.com	509-755-5393	39 Franklin Ave	Richland	None	
Nina Yaklin	nyaklin@hotmail.com	01434-621318	5679 Gloucester Pl	Calton Ward	cdaaburyfm	
Idella Scotland	idella@hotmail.com	08-7668-1355	373 Lafayette St	Cartmetcup	ajesus5	
Fausto Agramont	fausto_agramonte@yahoo.com	212-313-1783	5 Harrison Dr	New York	None	
Elvis Tenuta	elvis_tenuta@tenuta.org	403-415-1285	6437 Date Palm Dr	Calgary	mbyssouthq	
Maxima Brzezow	mbrzezowski@aol.com	905-569-6732	864 Augusti St	St Catharines	baucklandff	
Malcolm Gohlke	malcolm@yahoo.com	07-8026-3950	53247 Montgomery St #36	Southtown	abover56	
Ettie Luckenbach	ettie@yahoo.com	08-9378-7021	2902 Edison Dr #278	Mandurah East	tledsterbl	
Narcisca Araiza	narcisca@hotmail.com	01724-644767	8783 High St	Milton	thugonnetbi	
Dominique Dicker	dominique.dickerson@dickerson.org	510-983-3758	69 Marquette Ave	Hayward	None	

Delete

Insert

Top spending

First_name

Last_name

Address

City

Email

Phone

Username

Password

Submit

Customer screen: In here manager can see the overview information of customers and on

30

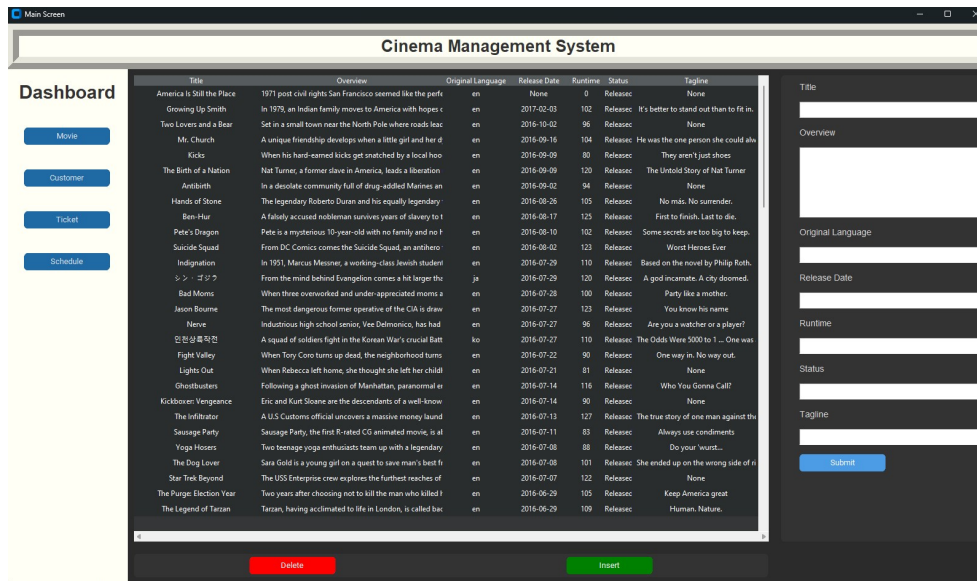
the right panel is where to inserting data. On the bottom is some function that helpful for manager to insert, delete and saw the Top spending.

ticket	username	schedule_movie_date	schedule_movie_start	room_name	cinema_name	movie_title	ticket_price	discount_value	seat_detail	seat_detail
5440	cuonghoang23	2024-12-28	04:20	Room_1	Xinema_1	Spider-Man 3	40000	10	A1_1 (00000)	A2_1 (00
5350	nhuromeqi	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	A2_1 (00000)	
5350	chollingdale27	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	A3_1 (00000)	
5350	permsonbv	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	A4_1 (00000)	
5350	mawicottjb	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	A5_1 (00000)	
5350	lqaurnd18	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	A6_1 (00000)	
5350	ccharles89	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	A7_1 (00000)	
5350	None	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	A8_1 (00000)	
5350	kdumr9	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	A9_1 (00000)	
5350	yhyromsly	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	A10_1 (00000)	
5350	lterbecklx	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	A11_1 (00000)	
5350	None	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	A12_1 (00000)	
5350	kwappletg	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	A13_1 (00000)	
5350	aathendfd	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	B1_1 (00000)	
5350	plafayd0	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	B2_1 (00000)	
5350	None	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	B3_1 (00000)	
5350	None	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	B4_1 (00000)	
5350	None	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	B5_1 (00000)	
5350	gbanepidd	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	B6_1 (00000)	
5350	lhtarnd8	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	B7_1 (00000)	
5350	None	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	B8_1 (00000)	
5350	None	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	B9_1 (00000)	
5350	gravesmicroftb6	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	B10_1 (00000)	
5350	kaybbecs	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	B11_1 (00000)	
5350	ginearsw	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	B12_1 (00000)	
5350	lseedman1x	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	B13_1 (00000)	
5350	nemdescottqr	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	C1_1 (00000)	
5350	scowderoyrc	2024-11-17	04:20	Room_1	Xinema_1	Spider-Man 3	80000	None	C2_1 (00000)	

Ticket screen: In here manager can have an overview of all ticket, and they can do some function like search the ticket, see the history transaction of customers, search the tickets by date and see number ticket sold per day of each movie.

Date	Start Time	Room	Cinema	Movie
2024-12-28	04:20	Room_1	Xinema_1	Spider-Man 3
2024-11-17	22:20	Room_10	Xinema_1	John Carter
2024-11-17	22:20	Room_9	Xinema_1	Spectre
2024-11-17	22:20	Room_8	Xinema_1	The Dark Knight Rises
2024-11-17	22:20	Room_7	Xinema_1	The Dark Knight Rises
2024-11-17	22:20	Room_6	Xinema_1	Avatar
2024-11-17	22:20	Room_5	Xinema_1	Pirates of the Caribbean: At World's End
2024-11-17	22:20	Room_4	Xinema_1	The Dark Knight Rises
2024-11-17	22:20	Room_3	Xinema_1	Avatar
2024-11-17	22:20	Room_2	Xinema_1	Tangled
2024-11-17	22:20	Room_1	Xinema_1	Spider-Man 3
2024-11-17	19:20	Room_10	Xinema_1	Spectre
2024-11-17	19:20	Room_9	Xinema_1	Tangled
2024-11-17	19:20	Room_8	Xinema_1	Tangled
2024-11-17	19:20	Room_7	Xinema_1	Spectre
2024-11-17	19:20	Room_6	Xinema_1	Spectre
2024-11-17	19:20	Room_5	Xinema_1	Pirates of the Caribbean: At World's End
2024-11-17	19:20	Room_4	Xinema_1	Spider-Man 3
2024-11-17	19:20	Room_3	Xinema_1	John Carter
2024-11-17	19:20	Room_2	Xinema_1	Spider-Man 3
2024-11-17	19:20	Room_1	Xinema_1	Pirates of the Caribbean: At World's End
2024-11-17	16:20	Room_10	Xinema_1	The Dark Knight Rises
2024-11-17	16:20	Room_9	Xinema_1	Spider-Man 3
2024-11-17	16:20	Room_8	Xinema_1	John Carter
2024-11-17	16:20	Room_7	Xinema_1	Tangled
2024-11-17	16:20	Room_6	Xinema_1	Avatar
2024-11-17	16:20	Room_5	Xinema_1	Avatar
2024-11-17	16:20	Room_4	Xinema_1	The Dark Knight Rises

Schedule screen: Manager can see all the schedule date at current and future day. They can insert by add the file csv with specific format, and they also can delete each schedule.



Movie screen: Manager can see all the movie and they can insert and delete each movie. For more details, access to [thisLink](#).

8 Result/Conclusion

8.1 Difficulties during Project Execution

1. With Teamwork

- Not having much time to discuss offline.
- Different timetables.
- Hard to express ideas by using natural language.
- Merge different parts done by each member into one complete project.

2. With Database Project

- Many problems occur when designing ERD and Relational Schema since they need to have many applications in reality.
- Collect sample data appropriately.
- Merge conflicts and inconsistency between components built by different members.
- We can't create proper automatic processing to booking ticket. When insert data into database, due to our creation, there a barrier to put data into ticket since we have to put data into schedule, seat, product firstly. So that we have not brought out this function into our interface of application. We will work together to find a solution to that problem in the future.

8.2 Lesson Learned

- Carefully analyze and thoroughly consider potential requirements and challenges before initiating any task or project.
- Possess the necessary knowledge to design and evaluate the ERD to prevent unnecessary revisions during the coding phase.
- Prioritize performance and incorporate the principles learned in class into the project.
- Adhere to essential steps and best practices when developing a project.
- Master query syntax and gain extensive experience in managing and executing projects.
- Acquire proficiency in implementing Python libraries to develop graphical user interfaces (GUIs).